

Introducción a Entity Framework

con el ecosistema .NET

Pontificia Universidad Católica del Ecuador – Manabí

Estudiantes: Steven Mero, Joshua Castañeda, Kevin Vera, Saul Velez, Alex Palma, Carlos Andrade

Docente: Ing. José Naranjo | Práctica en gestión de la información

Temario y Objetivos

Desarrollar sistemas comprendiendo la conexión mediante ADO.NET, la implementación de CRUD con Entity Framework, el uso de migraciones, el enfoque Code First y el poder de LINQ.

Fundamentos de ADO.NET



SqlConnection

Establece y administra la conexión entre la aplicación y SQL Server. Proporciona un canal seguro mediante una cadena de conexión. Gestiona los recursos (uso de `using`).



SqlCommand

Representa una instrucción SQL enviada al servidor (SELECT, INSERT, UPDATE, DELETE). Métodos principales: `ExecuteReader`, `ExecuteNonQuery` y `ExecuteScalar`.



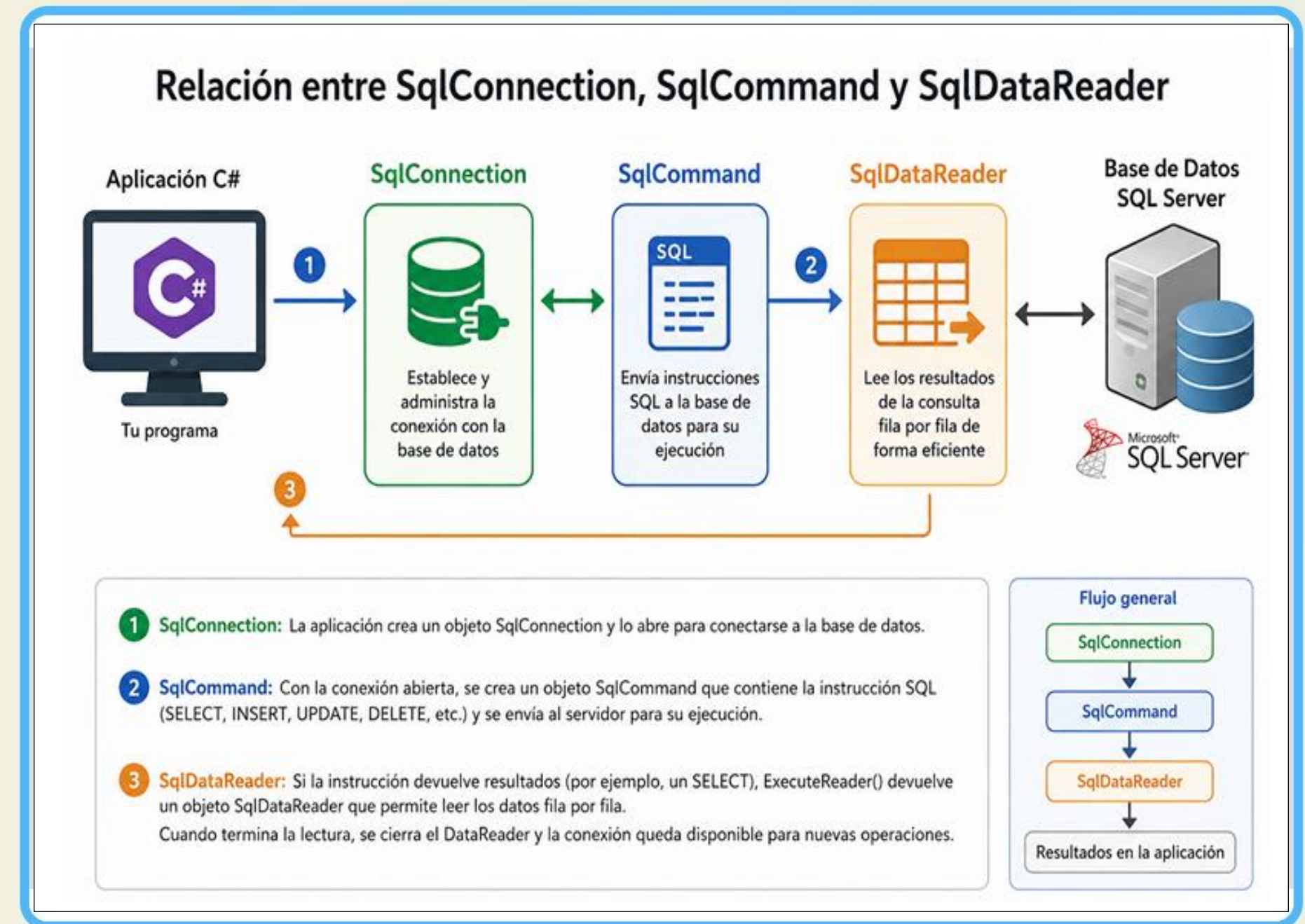
SqlDataReader

Lee resultados de forma secuencial y eficiente. Ideal para recorrer grandes registros fila por fila mediante `Read()`, reduciendo drásticamente el consumo de memoria.

Flujo de Conexión a Base de Datos

El acceso a datos con C# y .NET sigue un flujo estructurado y seguro:

- **1. Inicialización:** La aplicación crea un objeto `SqlConnection` y lo abre para conectarse a la base de datos.
- **2. Ejecución:** Con la conexión abierta, se crea un `SqlCommand` con la instrucción SQL y se envía al servidor.
- **3. Lectura:** Si devuelve resultados (ej. `SELECT`), `ExecuteReader()` devuelve un `SqlDataReader` para leer datos fila por fila.
- **4. Cierre:** Al finalizar, se cierra el `DataReader` y la conexión, liberando recursos.



Introducción a Entity Framework

Entity Framework Core es un mapeador objeto-relacional (ORM) moderno. Permite interactuar con bases de datos utilizando objetos C# tipados, eliminando código SQL repetitivo.



Modelos

Clases simples de C# que representan la estructura de datos. Cada propiedad de la clase suele corresponder a una columna en la tabla de la base de datos.



DbSet

Colección genérica que representa una tabla o vista. Es la puerta de entrada para realizar consultas y guardado para una entidad en particular.



DbContext

Componente central que representa una sesión con la BD. Gestiona conexiones, configura mapeos, rastrea cambios en objetos y coordina transacciones.

Análisis de Entity Framework

Ventajas

- **Abstracción de alto nivel:** No escribes SQL en cadenas de texto, evitando errores tipográficos.
- **Productividad:** Reduce drásticamente el código repetitivo.
- **Seguridad:** Consultas con LINQ se parametrizan automáticamente, previniendo inyección SQL.
- **Portabilidad:** Permite cambiar de motor de BD (ej. SQL Server a PostgreSQL) fácilmente.

Desventajas

- **Sobrecarga de rendimiento (Overhead):** Compilar consultas LINQ a SQL consume más CPU y memoria inicial.
- **Pérdida de control directo:** En consultas masivas o reportes muy complejos, el SQL generado puede no ser el óptimo.
- **Curva de aprendizaje:** Requiere comprender los contextos, ciclos de vida y el rastreo de entidades.

Migraciones en Entity Framework

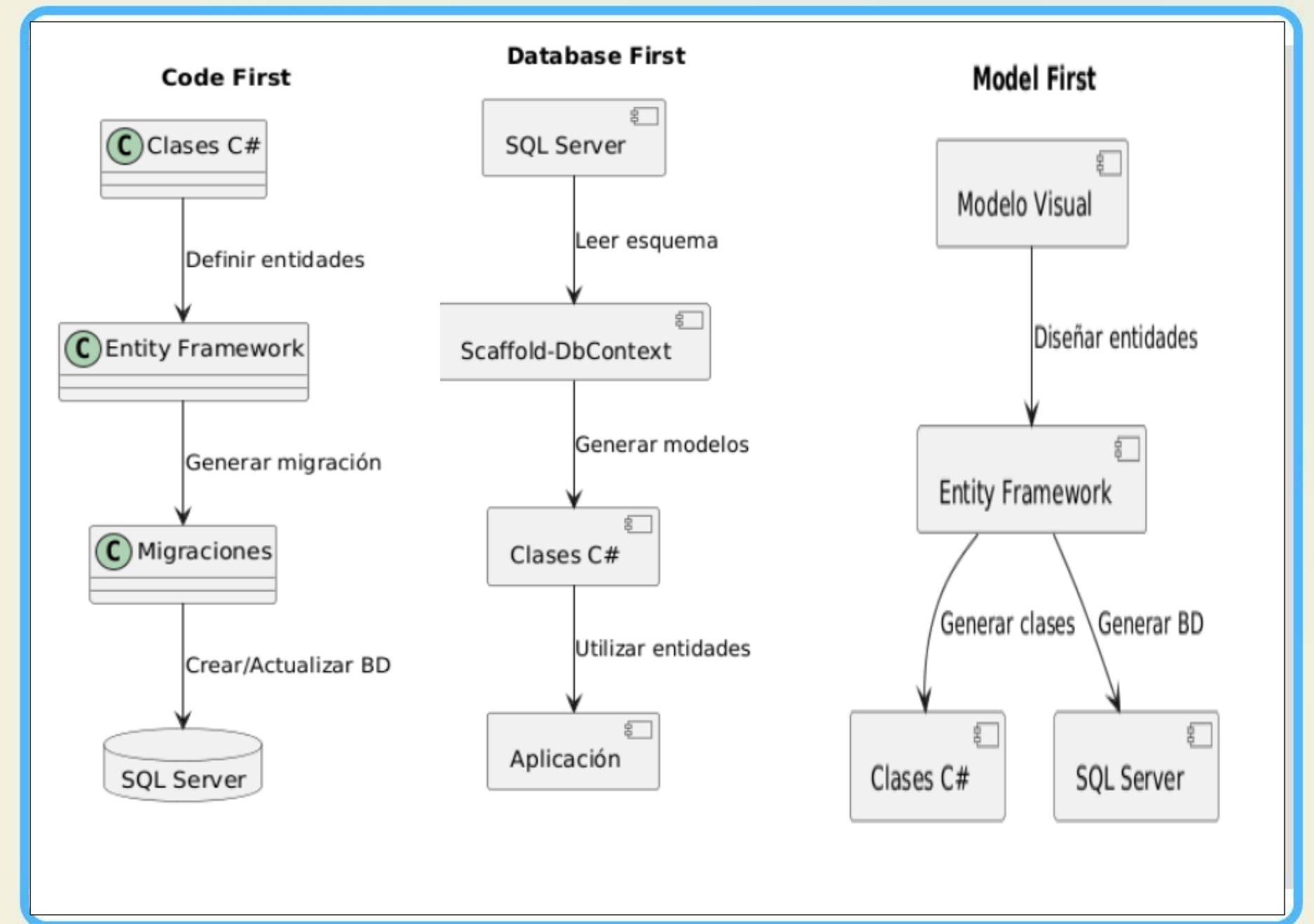
Sincronización Inteligente

Las migraciones permiten actualizar y mantener sincronizada la base de datos con los cambios realizados en las clases del proyecto (Modelos C#).

Cuando se agrega una propiedad, tabla o relación, Entity Framework genera automáticamente los scripts SQL necesarios para reflejar esos cambios.

Beneficios Clave:

- Control de versiones exhaustivo de la base de datos.
- Actualización automática del esquema relacional.
- Facilita el despliegue y trabajo concurrente en equipo.
- Evita la creación manual de scripts SQL propensos a errores.



Enfoques de Desarrollo en EF



Code First

Primero se crean las clases en C# y EF genera automáticamente la BD mediante migraciones. Ideal para proyectos nuevos por su desarrollo rápido y alto control desde el código.



Database First

Primero se crea la base de datos (o se usa una existente) y EF genera automáticamente las clases C#. Excelente para integrar sistemas heredados empresariales.



Model First

Se diseña un modelo gráfico visual (ER) y de ahí se genera la BD y las clases. Tiene un diseño intuitivo pero es menos flexible y actualmente menos utilizado.

Comparación de Enfoques

Característica	Code First	Database First	Model First
Punto de inicio	Código (Clases C#)	Base de datos	Modelo gráfico
Uso de migraciones	Sí	Limitado	Sí
Ideal para proyectos nuevos	Sí	No	Sí
Ideal para BD existente	No	Sí	No
Facilidad de mantenimiento	Alta	Media	Media
Popularidad actual	Muy alta (Recomendado)	Alta	Baja

Entity Framework vs ADO.NET

Aspecto	ADO.NET	Entity Framework (Core)
Abstracción	Baja (SQL explícito)	Alta (LINQ a SQL)
Curva de aprendizaje	Moderada (requiere saber SQL a fondo)	Mayor al inicio, luego mucho más ágil
Rendimiento	Máximo control, mínimo overhead	Ligero overhead; EF Core es muy eficiente
Productividad	Más código boilerplate y manual	Mucho menos código, refactoring seguro
Mantenimiento	Propenso a errores en strings SQL	Fuertemente tipado en tiempo de compilación
Migraciones / DB	Manual / Dependiente del driver	Automáticas / Multi-proveedor de BD

¿Cuál elegir en tus proyectos?



Cuándo usar ADO.NET:

Cuando tienes rendimiento extremadamente crítico (ej. consultas con millones de filas, procesos ETL batch). También es ideal para ejecutar sentencias SQL muy complejas (PIVOT, CTEs recursivos, hints de índices) o construir microservicios ultra-ligeros.



Cuándo usar Entity Framework Core:

Perfecto para aplicaciones nuevas de negocio, CRUD estándar, iteración rápida y creación de prototipos. Ayuda en equipos donde no todos dominan SQL, utilizando arquitecturas DDD o de dominio rico.



Regla Práctica Actual:

En proyectos modernos, **EF Core cubre el 90% de los casos** con mucha menos fricción. ADO.NET sigue siendo indispensable solo cuando cada milisegundo y cada byte importan absolutamente.

¿Preguntas?

Gracias por su atención a la presentación.